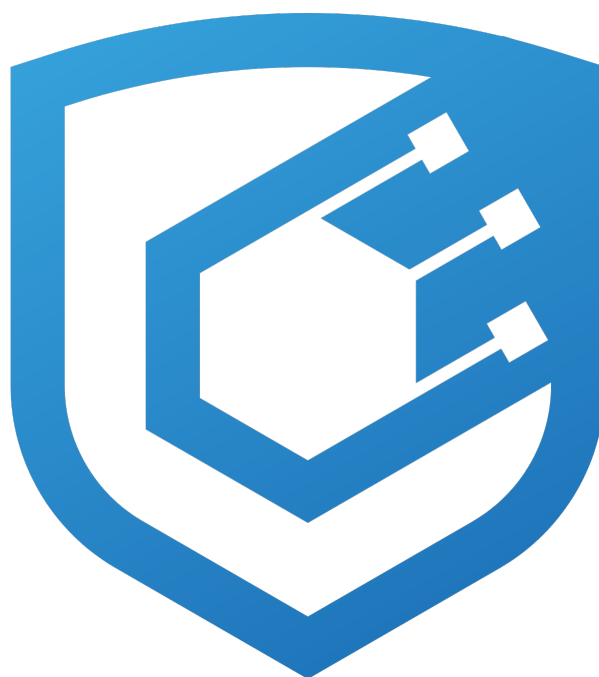




---

# BTCY Audit Report

---

Prepared by [Cyfrin](#)

Version 2.0

## Lead Auditors

[Dacian](#)

[Viktor](#)

February 20, 2026

# Contents

|          |                                                                                                                                                             |          |
|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| <b>1</b> | <b>About Cyfrin</b>                                                                                                                                         | <b>3</b> |
| <b>2</b> | <b>Disclaimer</b>                                                                                                                                           | <b>3</b> |
| <b>3</b> | <b>Risk Classification</b>                                                                                                                                  | <b>3</b> |
| <b>4</b> | <b>Protocol Summary</b>                                                                                                                                     | <b>3</b> |
| <b>5</b> | <b>Audit Scope</b>                                                                                                                                          | <b>3</b> |
| <b>6</b> | <b>Executive Summary</b>                                                                                                                                    | <b>4</b> |
| <b>7</b> | <b>Findings</b>                                                                                                                                             | <b>8</b> |
| 7.1      | Medium Risk                                                                                                                                                 | 8        |
| 7.1.1    | Missing or inadequate Chainlink Oracle checks in Pricer.sol                                                                                                 | 8        |
| 7.2      | Low Risk                                                                                                                                                    | 9        |
| 7.2.1    | Use Foundry's encrypted secure private key storage instead of plaintext environment variables                                                               | 9        |
| 7.2.2    | Fees should round up in favor of the protocol                                                                                                               | 9        |
| 7.2.3    | ERC4626 vaults must override maxWithdraw, maxDeposit, maxMint, maxRedeem to return zero when paused or for user-specific limits such as being denied access | 9        |
| 7.2.4    | Batch processing can be DoS'd by cancellation/rescue of a single included request ID                                                                        | 9        |
| 7.2.5    | Incorrect storage location constants IBTCYHub::IBTCYHUB_STORAGE_LOCATION and Pricer::PRICER_STORAGE_LOCATION                                                | 10       |
| 7.2.6    | IBTCY::processSubscriptions doesn't revert when subscriptionPaused == true                                                                                  | 11       |
| 7.2.7    | Pricer::updatePrice allows infinite price changes in the same block                                                                                         | 11       |
| 7.2.8    | IBTCYHub batch processing uses historical Chainlink rounds                                                                                                  | 11       |
| 7.2.9    | IBTCY::burn, forceTransfer can burn IBTCY tokens held by the BTCY vault, breaking the 1-to-1 collateral backing                                             | 11       |
| 7.2.10   | IBTCY::approve is blocked while paused, preventing users from revoking allowances during emergencies                                                        | 11       |
| 7.2.11   | Decimals mismatch risk for supported collateral can cause catastrophic over-mints                                                                           | 12       |
| 7.2.12   | Pricer silently truncates priceIds to uint80 which could map to incorrect Chainlink rounds                                                                  | 12       |
| 7.3      | Informational                                                                                                                                               | 13       |
| 7.3.1    | Use named mappings to explicitly indicate the purpose of keys and values                                                                                    | 13       |
| 7.3.2    | Misleading event emission when previous state not changed                                                                                                   | 13       |
| 7.3.3    | ERC4626::deposit, redeem, mint, withdraw should revert if they would return zero                                                                            | 13       |
| 7.3.4    | Consider enforcing minimum deposit amount                                                                                                                   | 14       |
| 7.3.5    | Use the transient keyword instead of low-level assembly                                                                                                     | 14       |
| 7.3.6    | Consider automatic allowlist configuration when changing compliance, fee receiver address                                                                   | 14       |
| 7.3.7    | Rename Deposit::amountDepositedMinusFees since subscription fee only used to emit event but not subtracted from amount deposited                            | 14       |
| 7.3.8    | IBTCYHub::_processSubscription should follow Checks-Effects-Interactions pattern by updating depositor amount before making external call to mint tokens    | 15       |
| 7.3.9    | Remove obsolete return statements when already using named return variables                                                                                 | 15       |
| 7.3.10   | Duplicate fee helper logic in IBTCYHub can be consolidated                                                                                                  | 16       |
| 7.3.11   | Inconsistent naming of IBTCYHub::_processRedemption return value                                                                                            | 16       |
| 7.3.12   | DepositWithdraw, IBTCYHub, Pricer allow only default admin to revoke role in contrast to the other contracts                                                | 16       |
| 7.3.13   | Fee on transfer tokens not supported                                                                                                                        | 16       |
| 7.3.14   | Consider enhancing BTCY::_checkIBTCYTransfer to also prevent transfers to address(this)                                                                     | 16       |
| 7.3.15   | IBTCY::_checkBTCYVaultTransfer bypassed for mint                                                                                                            | 16       |
| 7.3.16   | nonReentrant is not the first modifier on IBTCYHub::processSubscriptions / processRedemptions                                                               | 17       |
| 7.3.17   | Zero amount redemption requests can't be cancelled or processed                                                                                             | 17       |
| 7.3.18   | Remove unused errors                                                                                                                                        | 17       |

|        |                                                                                                     |    |
|--------|-----------------------------------------------------------------------------------------------------|----|
| 7.3.19 | Enforce BTCY vault decimals greater or equal to underlying token decimals . . . . .                 | 18 |
| 7.3.20 | BTCY::_checkMinShares should not allow zero totalSupply as it is called after deposits . .          | 18 |
| 7.3.21 | User can front-run DoS DepositWithdraw::withdrawTransfer by cancelling redemption request . . . . . | 18 |
| 7.4    | Gas Optimization . . . . .                                                                          | 20 |
| 7.4.1  | In Solidity don't initialize to default values . . . . .                                            | 20 |
| 7.4.2  | Use ReentrancyGuardTransient for faster nonReentrant modifiers . . . . .                            | 20 |
| 7.4.3  | Optimize away old value variables when emitting events by emitting events first . . . . .           | 20 |
| 7.4.4  | Use named return variables where this can optimize away local variables . . . . .                   | 21 |
| 7.4.5  | Remove redundant modifier from BTCY, IBTCY::revokeRole . . . . .                                    | 21 |
| 7.4.6  | Cache storage to avoid identical storage reads . . . . .                                            | 21 |
| 7.4.7  | Better storage packing . . . . .                                                                    | 22 |
| 7.4.8  | Redundant DEFAULT_ADMIN_ROLE grant in AllowList::_AllowList_init . . . . .                          | 22 |
| 7.4.9  | Remove redundant whenNotPaused modifiers . . . . .                                                  | 22 |
| 7.4.10 | Remove event parameters that always have the same value . . . . .                                   | 22 |
| 7.4.11 | More efficient implementations of IBTCYHub::processSubscriptions, processRedemptions                | 23 |
| 7.4.12 | Remove empty function BTCY::_checkMinSharesAfterWithdraw . . . . .                                  | 26 |

# 1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at [cyfrin.io](https://cyfrin.io).

## 2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the solidity implementation of the contracts.

## 3 Risk Classification

|                    | Impact: High | Impact: Medium | Impact: Low |
|--------------------|--------------|----------------|-------------|
| Likelihood: High   | Critical     | High           | Medium      |
| Likelihood: Medium | High         | Medium         | Low         |
| Likelihood: Low    | Medium       | Low            | Low         |

## 4 Protocol Summary

BTCY is an institutional two-token framework for Bitcoin yield, featuring ERC20/ERC4626 compliance, upgradeable architecture, and comprehensive institutional controls. The protocol consists of five core contracts:

- IBTCY : ERC-20 token with allowlist enforcement, pause functionality, compliance-driven force transfers, and vault protection. Uses transient storage for force transfer state management
- BTCY : ERC-4626 vault wrapper around iBTCY with withdrawal fees, transfer deny list, minimum deposit/withdraw amounts, and donation attack protection via MIN\_SHARES
- IBTCYHub : subscription and redemption hub managing the lifecycle of deposit and redemption requests, with fee bounds, pause controls, and price-based processing
- Pricer : price oracle contract that stores price entries with Chainlink validation, rate limiting, deviation bounds, and sequencer uptime checks
- DepositWithdraw : treasury deposit/withdraw contract with token and user whitelisting, pause controls, and role-based authorization

All contracts use OpenZeppelin's upgradeable access control (`AccessControlEnumerableUpgradeable`) except `DepositWithdraw` which uses non-upgradeable `AccessControlEnumerable`; storage for upgradeable contracts follows the ERC-7201 namespaced pattern.

## 5 Audit Scope

The audit scope was limited to:

```
src/base/AllowList.sol
src/BTCY.sol
src/DepositWithdraw.sol
src/IBTCY.sol
src/IBTCYHub.sol
```

The client made some additional changes during the audit including to the deployment scripts which we reviewed and provided some feedback on.

## 6 Executive Summary

Over the course of 6 days, the Cyfrin team conducted an audit on the [BTCY](#) smart contracts provided by [Aarc](#). In this period, a total of 46 issues were found.

The findings consist of 1 Medium and 12 Low severity issues with the remainder being informational and gas optimizations.

A formal verification suite was created and provided as an additional deliverable with the audit.

### Decimal Mismatch Off-Chain Component Risks

During the audit we noted that comments and constants in the code appeared to assume that all collateral tokens will have 8 decimal precision (see finding L-12), but that from the listed collateral `tBTC` uses 18 decimal precision.

As the protocol has off-chain components that were outside the scope of this audit, we recommend careful testing of the off-chain components ensuring they correctly handle 18 decimal precision tokens and perform any required conversion before/after interacting with the smart contracts.

### Centralization Risks

The protocol by its regulated nature is highly centralized with protocol admins having complete power including the ability to upgrade most contracts; protocol users must place complete trust in the protocol admins when engaging with the protocol. One benefit to this is that even if a serious bug should be discovered once the protocol is live, protocol admins will in many cases be able to upgrade the relevant contracts and resolve the issue.

### Post Audit Changes

After the audit had concluded we were asked to examine some changes to the deployment scripts and the addition of a new function `DepositWithdraw::withdrawTransfer` in [PR20](#); we provided some feedback on these post-audit changes.

### Summary

|                |                                     |
|----------------|-------------------------------------|
| Project Name   | BTCY                                |
| Repository     | <a href="#">btcy-contracts-main</a> |
| Commit         | <a href="#">d95495e418c3...</a>     |
| Fix Commit     | <a href="#">5040483a51a1...</a>     |
| Audit Timeline | Feb 9th - Feb 16th, 2026            |
| Methods        | Manual Review, Formal Verification  |

### Issues Found

|                   |    |
|-------------------|----|
| Critical Risk     | 0  |
| High Risk         | 0  |
| Medium Risk       | 1  |
| Low Risk          | 12 |
| Informational     | 21 |
| Gas Optimizations | 12 |
| Total Issues      | 46 |

### Summary of Findings

|                                                                                                                                                                                                                           |              |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|
| [M-1] Missing or inadequate Chainlink Oracle checks in <code>Pricer.sol</code>                                                                                                                                            | Resolved     |
| [L-01] Use Foundry's encrypted secure private key storage instead of plaintext environment variables                                                                                                                      | Resolved     |
| [L-02] Fees should round up in favor of the protocol                                                                                                                                                                      | Resolved     |
| [L-03] ERC4626 vaults must override <code>maxWithdraw</code> , <code>maxDeposit</code> , <code>maxMint</code> , <code>maxRedeem</code> to return zero when paused or for user-specific limits such as being denied access | Resolved     |
| [L-04] Batch processing can be DoS'd by cancellation/rescue of a single included request ID                                                                                                                               | Resolved     |
| [L-05] Incorrect storage location constants <code>IBTCYHub::IBTCYHUB_STORAGE_LOCATION</code> and <code>Pricer::PRICER_STORAGE_LOCATION</code>                                                                             | Resolved     |
| [L-06] <code>IBTCY::processSubscriptions</code> doesn't revert when <code>subscriptionPaused == true</code>                                                                                                               | Resolved     |
| [L-07] <code>Pricer::updatePrice</code> allows infinite price changes in the same block                                                                                                                                   | Resolved     |
| [L-08] IBTCYHub batch processing uses historical Chainlink rounds                                                                                                                                                         | Acknowledged |
| [L-09] <code>IBTCY::burn</code> , <code>forceTransfer</code> can burn IBTCY tokens held by the BTCY vault, breaking the 1-to-1 collateral backing                                                                         | Resolved     |
| [L-10] <code>IBTCY::approve</code> is blocked while paused, preventing users from revoking allowances during emergencies                                                                                                  | Resolved     |
| [L-11] Decimals mismatch risk for supported collateral can cause catastrophic over-mints                                                                                                                                  | Resolved     |
| [L-12] <code>Pricer</code> silently truncates <code>priceIds</code> to <code>uint80</code> which could map to incorrect Chainlink rounds                                                                                  | Resolved     |
| [I-01] Use named mappings to explicitly indicate the purpose of keys and values                                                                                                                                           | Resolved     |
| [I-02] Misleading event emission when previous state not changed                                                                                                                                                          | Resolved     |
| [I-03] <code>ERC4626::deposit</code> , <code>redeem</code> , <code>mint</code> , <code>withdraw</code> should revert if they would return zero                                                                            | Resolved     |
| [I-04] Consider enforcing minimum deposit amount                                                                                                                                                                          | Resolved     |

|                                                                                                                                                                              |              |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|
| [I-05] Use the <code>transient</code> keyword instead of low-level assembly                                                                                                  | Resolved     |
| [I-06] Consider automatic allowlist configuration when changing compliance, fee receiver addresse                                                                            | Acknowledged |
| [I-07] Rename <code>Deposit::amountDepositedMinusFees</code> since subscription fee only used to emit event but not subtracted from amount deposited                         | Resolved     |
| [I-08] <code>IBTCYHub::_processSubscription</code> should follow Checks-Effects-Interactions pattern by updating depositor amount before making external call to mint tokens | Resolved     |
| [I-09] Remove obsolete <code>return</code> statements when already using named return variables                                                                              | Resolved     |
| [I-10] Duplicate fee helper logic in <code>IBTCYHub</code> can be consolidated                                                                                               | Resolved     |
| [I-11] Inconsistent naming of <code>IBTCYHub::_processRedemption</code> return value                                                                                         | Resolved     |
| [I-12] <code>DepositWithdraw</code> , <code>IBTCYHub</code> , <code>Pricer</code> allow only default admin to revoke role in contrast to the other contracts                 | Resolved     |
| [I-13] Fee on transfer tokens not supported                                                                                                                                  | Acknowledged |
| [I-14] Consider enhancing <code>BTCY::_checkIBTCYTransfer</code> to also prevent transfers to <code>address(this)</code>                                                     | Resolved     |
| [I-15] <code>IBTCY::_checkBTCYVaultTransfer</code> bypassed for mint                                                                                                         | Acknowledged |
| [I-16] <code>nonReentrant</code> is not the first modifier on <code>IBTCYHub::processSubscriptions / processRedemptions</code>                                               | Resolved     |
| [I-17] Zero amount redemption requests can't be cancelled or processed                                                                                                       | Resolved     |
| [I-18] Remove unused errors                                                                                                                                                  | Resolved     |
| [I-19] Enforce BTCY vault decimals greater or equal to underlying token decimals                                                                                             | Resolved     |
| [I-20] <code>BTCY::_checkMinShares</code> should not allow zero <code>totalSupply</code> as it is called after deposits                                                      | Resolved     |
| [I-21] User can front-run DoS <code>DepositWithdraw::withdrawTransfer</code> by cancelling redemption request                                                                | Acknowledged |
| [G-01] In Solidity don't initialize to default values                                                                                                                        | Resolved     |
| [G-02] Use <code>ReentrancyGuardTransient</code> for faster <code>nonReentrant</code> modifiers                                                                              | Resolved     |
| [G-03] Optimize away old value variables when emitting events by emitting events first                                                                                       | Resolved     |
| [G-04] Use named return variables where this can optimize away local variables                                                                                               | Resolved     |
| [G-05] Remove redundant modifier from <code>BTCY</code> , <code>IBTCY::revokeRole</code>                                                                                     | Resolved     |
| [G-06] Cache storage to avoid identical storage reads                                                                                                                        | Resolved     |
| [G-07] Better storage packing                                                                                                                                                | Resolved     |
| [G-08] Redundant <code>DEFAULT_ADMIN_ROLE</code> grant in <code>AllowList::_AllowList_init</code>                                                                            | Resolved     |
| [G-09] Remove redundant <code>whenNotPaused</code> modifiers                                                                                                                 | Resolved     |

|                                                                                             |          |
|---------------------------------------------------------------------------------------------|----------|
| [G-10] Remove event parameters that always have the same value                              | Resolved |
| [G-11] More efficient implementations of IBTCYHub::processSubscriptions, processRedemptions | Resolved |
| [G-12] Remove empty function BTCY::_checkMinSharesAfterWithdraw                             | Resolved |

## 7 Findings

### 7.1 Medium Risk

#### 7.1.1 Missing or inadequate Chainlink Oracle checks in `Pricer.sol`

**Description:** `Pricer.sol` is missing or has inadequate [chainlink oracle checks](#):

- protocol intends to deploy on L2s explicitly Arbitrum & Base but has no checks for [L2 sequencer downtime](#) - when implementing this also [revert](#) if `startedAt == 0`
- no enforcement that returned chainlink price is inside the [aggregator's min/max price](#)

Also consider that the `Pricer` contract currently hard-codes:

- constants related to Chainlink price feed heart-beats
- `CHAINLINK_PRICE_SCALE` which assumes the Chainlink price feed has 8 decimals (as does constant `IBTCY-Hub::DECIMALS_MULTIPLIER`)

Both of these assumptions are not always true for all Chainlink price feeds. Consider whether these should be configurable parameters or at least when deploying new instances of `Pricer`, verify that these constants are correct for the intended price-feed and modify them if necessary.

Finally also consider [risks of depeg](#) and whether the protocol needs to detect for a potential depeg event with the intended tokens being used.

**Aarc:** Fixed in [PR10](#).

**Cyfrin:** Verified.

## 7.2 Low Risk

### 7.2.1 Use Foundry's encrypted secure private key storage instead of plaintext environment variables

**Description:** The deployment scripts expect private keys to be available plaintext in environment variables. Instead consider [using](#) Foundry's encrypted secure private key storage.

**Aarc:** Fixed in commit [cc91182](#).

**Cyfrin:** Verified.

### 7.2.2 Fees should round up in favor of the protocol

**Description:** Fees should round up in favor of the protocol:

- `BTCY::_calculateWithdrawalFee`
- `IBTCYHub::_getRedemptionFees, _getSubscriptionFees`

**Recommended Mitigation:** Use OZ [mulDiv](#) with explicit rounding.

It has become best practice to always:

- use explicit rounding directions
- comment each with the logical reason for why the rounding direction is correct in that place

This forces developers to carefully consider each rounding direction which is important since incorrect rounding directions have frequently been part of mainnet hack exploit chains.

**Aarc:** Fixed in commit [b603dfa](#).

**Cyfrin:** Verified.

### 7.2.3 ERC4626 vaults must override `maxWithdraw`, `maxDeposit`, `maxMint`, `maxRedeem` to return zero when paused or for user-specific limits such as being denied access

**Description:** [EIP-4626](#) states on `maxDeposit`:

MUST factor in both global and user-specific limits, like if deposits are entirely disabled (even temporarily) it MUST return 0.

Similar statements in the EIP-4626 standard can be found related to `maxWithdraw`, `maxMint`, `maxRedeem`. If pausing or other limits are not accounted for by these functions this violates EIP-4626 which can break integrating protocols expecting standards-compliant behavior.

In the context of `BTCY`, these functions should return zero when:

- contract is paused
- `isTransferDenylisted(user) == true` (see check in `_update`)
- for `maxWithdraw`, `maxRedeem` when `isAllowlisted(user) == false` and when the withdrawn amount would be smaller than `$.minWithdrawAmount` (see checks in `_withdraw`)

**Aarc:** Fixed in commit [ac4f562](#), [2e39ebf](#).

**Cyfrin:** Verified.

### 7.2.4 Batch processing can be DoS'd by cancellation/rescue of a single included request ID

**Description:** The Hub's batch processing functions (`IBTCYHub::processRedemptions` and `IBTCYHub::processSubscriptions`) iterate over arrays of request IDs and revert if any entry is invalid (e.g., the stored user is `address(0)`). Since requests can be removed via cancellation (and redemptions can also be removed via rescue), a request ID can become invalid after an operator has constructed a batch but before the batch transaction executes. Including that now-invalid ID causes the entire batch call to revert.

**Impact:** A single cancelled/rescued request can block processing of other valid requests included in the same batch, disrupting automated settlement pipelines and delaying unrelated subscriptions/redemptions until the batch is rebuilt and resubmitted.

**Recommended Mitigation:** Make batch processing tolerant to invalid/missing entries (skip + optionally emit an event), or provide single-item processing functions so operators can avoid batch-wide failure when individual requests are cancelled/rescued.

**Aarc Btcy:** Fixed in [ce0eb1e](#)

**Cyfrin:** Verified. Zero address entries now are skipped instead of reverting along with an event.

## 7.2.5 Incorrect storage location constants IBTCYHub::IBTCYHUB\_STORAGE\_LOCATION and Pricer::PRICER\_STORAGE\_LOCATION

**Description:** The constant IBTCYHub::IBTCYHUB\_STORAGE\_LOCATION does not match its described source of value:

```
// keccak256(abi.encode(uint256(keccak256("ibtcy.storage.IBTCYHubStorage")) - 1)) &
↳ ~bytes32(uint256(0xff))
bytes32 private constant IBTCYHUB_STORAGE_LOCATION =
    0xb369edd2fa44207007022375c77d052217134e7a2010f2accab4bcd15e2a1800;

$ chisel
Welcome to Chisel! Type `!help` to show available commands.
keccak256(abi.encode(uint256(keccak256("ibtcy.storage.IBTCYHubStorage")) - 1)) &
↳ ~bytes32(uint256(0xff))
Type: uint256
Hex: 0x885b064fbe39cd82ef7bf8899062b705be9d48a0e61139bd1237502a5b46bc00
Hex (full word): 0x885b064fbe39cd82ef7bf8899062b705be9d48a0e61139bd1237502a5b46bc00
Decimal: 61675374050566043342209562594250077921514213409057574218735818105148427123712
```

The same is true for Pricer::PRICER\_STORAGE\_LOCATION:

```
// keccak256(abi.encode(uint256(keccak256("ibtcy.storage.PricerStorage")) - 1)) &
↳ ~bytes32(uint256(0xff))
// Note: Storage location hash kept for upgrade compatibility with deployed contracts
bytes32 private constant PRICER_STORAGE_LOCATION =
    0x85965ed6e8199ce504ed8450ed69152a88bd527f8a57daecbc8e1119dedaf000;

$ chisel
Welcome to Chisel! Type `!help` to show available commands.
keccak256(abi.encode(uint256(keccak256("ibtcy.storage.PricerStorage")) - 1)) & ~bytes32(uint256(0xff))
Type: uint256
Hex: 0x67c492fb5d80e67b32c4d1be5ccac32c2bd7136dceacb7de20ba7a8254426600
Hex (full word): 0x67c492fb5d80e67b32c4d1be5ccac32c2bd7136dceacb7de20ba7a8254426600
Decimal: 46935539860533315848262906699558992907666370328233995352751199683037920126464
```

**Recommended Mitigation:** The fix is only for new deployments not for upgrading existing deployments:

```
bytes32 private constant IBTCYHUB_STORAGE_LOCATION =
- 0xb369edd2fa44207007022375c77d052217134e7a2010f2accab4bcd15e2a1800;
+ 0x885b064fbe39cd82ef7bf8899062b705be9d48a0e61139bd1237502a5b46bc00;

bytes32 private constant PRICER_STORAGE_LOCATION =
- 0x85965ed6e8199ce504ed8450ed69152a88bd527f8a57daecbc8e1119dedaf000;
+ 0x67c492fb5d80e67b32c4d1be5ccac32c2bd7136dceacb7de20ba7a8254426600
```

**Aarc:** Fixed in commit [15a2e8c](#).

**Cyfrin:** Verified.

### 7.2.6 IBTCY::processSubscriptions **doesn't revert when** subscriptionPaused == true

**Description:** IBTCY::processSubscriptions doesn't revert when subscriptionPaused == true. This appears to be an oversight since its sister-function processRedemptions reverts when redemptionPaused == true.

**Aarc:** Fixed in commit [c129022](#).

**Cyfrin:** Verified.

### 7.2.7 Pricer::updatePrice **allows infinite price changes in the same block**

**Description:** Pricer::updatePrice does not contain any rate-limiting check such as addPrice, addCurrentPrice which use MIN\_PRICE\_UPDATE\_INTERVAL.

It does have a call to \_requireWithinDeviation but this compares the new price against the current price for the same priceId, and hence this is meaningless since PRICE\_UPDATE\_ROLE can just call updatePrice in the same block over and over again, increasing or decreasing the price by the max deviation amount each time.

**Recommended Mitigation:** Implement a time-based rate-limiting restriction in Pricer::updatePrice such that PRICE\_UPDATE\_ROLE can't call it for the same priceId multiple times in the same block.

**Aarc:** Fixed in commit [c905cbf](#).

**Cyfrin:** Verified.

### 7.2.8 IBTCYHub **batch processing uses historical Chainlink rounds**

**Description:** IBTCYHub::processSubscriptions/processRedemptions accept priceIds and fetch pricing via pricer.getPriceInfos(priceIds), which calls priceFeed.getRoundData(roundId) for each provided id. This uses historical Chainlink rounds rather than the latest available round at processing time.

**Impact:** Batches may be processed using prices that are older than necessary.

**Recommended Mitigation:** Consider using latestRoundData and skip (priceIds/roundIds).

**Aarc:** Acknowledged. Design choice.

### 7.2.9 IBTCY::burn, forceTransfer **can burn IBTCY tokens held by the BTCY vault, breaking the 1-to-1 collateral backing**

**Description:** BTCY is "an ERC-4626 vault token that wraps iBTCY on a one-to-one basis. Only holders of iBTCY may mint or redeem BTCY, ensuring that BTCY remains fully collateralized".

However IBTCY::burn, forceTransfer can burn IBTCY tokens held by the BTCY vault, breaking the 1-to-1 collateral backing.

**Recommended Mitigation:** Revert in IBTCY::burn, forceTransfer if the target is the IBTCY vault. For example:

```
function burn(address from, uint256 amount) public override onlyRole(BURNER_ROLE) {
+   require(from != _getIBTCYStorage().btcVault, CannotBurnFromVault());
    _burn(from, amount);
}
```

**Aarc:** Fixed in commit [9089464](#).

**Cyfrin:** Verified.

### 7.2.10 IBTCY::approve **is blocked while paused, preventing users from revoking allowances during emergencies**

**Description:** IBTCY::approve has the whenNotPaused modifier. In a pause scenario, users cannot update allowances, including reducing them to zero to revoke previously granted approvals. Since transfers are already

restricted under pause, allowing allowance updates during pause would not meaningfully expand token movement but would let users reduce exposure to compromised spenders.

**Impact:** During an incident (e.g., compromised spender/contract), pausing the token prevents users from revoking approvals, leaving existing allowances in place until unpaused and increasing risk if/when the token is later unpaused or if any allowance-based pathway remains usable.

**Recommended Mitigation:** Consider removing the `whenNotPaused` restriction from `approve`.

**Aarc:** Fixed in commit [e52faa9](#).

**Cyfrin:** Verified.

### 7.2.11 Decimals mismatch risk for supported collateral can cause catastrophic over-mints

**Description:** The conversion logic in `IBTCYHub` assumes input amounts are expressed in an 8-decimal “BTC base unit”, as reflected by the fixed scaling in both `IBTCYHub::_getMintAmountForPrice` and `IBTCYHub::_getRedemptionAmountForRwa`. If an 18-decimal collateral such as `tBTC` is supported and raw token units are mistakenly used (or any offchain pipeline fails to downscale 18→8 before calling the Hub), the calculations will treat 18-decimal units as 8-decimal units.

**Impact:** An 18→8 decimal mis-scaling can cause incorrect minting and redemption settlement by approximately  $10^{(18-8)} = 1e10$  for the same human-denominated collateral amount, leading to severe over-issuance and/or incorrect payout amounts from a single operational error.

**Recommended Mitigation:** Add onchain sanity bounds to prevent catastrophic mis-scaling, such as:

- a configurable maximum per-subscription “BTC base unit” amount (expected 8-decimal normalized input), and/or
- a configurable maximum `IBTCY` mint amount per request/batch, chosen to accommodate normal operational sizes while rejecting obviously mis-scaled inputs. This provides defense-in-depth without requiring the Hub to know the underlying collateral token decimals.

Additionally, explicitly state in the function/interface documentation that all subscription/redemption amount parameters are expected to be provided in 8-decimal BTC base units even if the collateral token is 18 decimal.

**Aarc:** Fixed in [3ffa5a4](#).

**Cyfrin:** Verified. A max amount for both subscriptions and redemptions is now enforced.

### 7.2.12 Pricer silently truncates priceIds to uint80 which could map to incorrect Chainlink rounds

**Description:** Pricer silently truncates `priceIds` to `uint80` which could map to incorrect Chainlink rounds:

```
Pricer.sol
161:         (, int256 answer,, uint256 updatedAt,) = $.priceFeed.getRoundData(uint80(priceId));
184:         (, int256 answer,, uint256 updatedAt,) = $.priceFeed.getRoundData(uint80(priceId));
211:         (, int256 answer,, uint256 updatedAt,) = $.priceFeed.getRoundData(uint80(_priceIds[i]));
```

**Recommended Mitigation:** Consider using `SafeCast::toUint80` to revert in such cases.

**Aarc:** Fixed in commit [48598d4](#).

**Cyfrin:** Verified.

## 7.3 Informational

### 7.3.1 Use named mappings to explicitly indicate the purpose of keys and values

**Description:** Use named mappings to explicitly indicate the purpose of keys and values:

```
base/AllowList.sol
26:      mapping(address => bool) allowlistedAddresses;

Pricer.sol
57:      mapping(uint256 => PriceInfo) prices;

IBTCYHub.sol
104:     mapping(bytes32 => Depositor) depositIdToDepositor;
106:     mapping(bytes32 => Redeemer) redemptionIdToRedeemer;

BTCY.sol
56:      mapping(address => bool) transferDenylisted;

DepositWithdraw.sol
50:      mapping(address => bool) private _whitelistedTokens;
53:      mapping(address => bool) private _whitelistedUsers;
```

**Aarc:** Fixed in commit [50e7b72](#).

**Cyfrin:** Verified.

### 7.3.2 Misleading event emission when previous state not changed

**Description:** There is an asymmetry in that `AllowList::removeFromAllowlist` doesn't check whether address exists while `batchRemoveFromAllowlist` does.

This means that `removeFromAllowlist` will emit `AddressRemovedFromAllowlist` even though the account was not removed.

**Recommended Mitigation:** `AllowList::removeFromAllowlist` should only emit the `AddressRemovedFromAllowlist` if the account was actually removed similar to `batchRemoveFromAllowlist`.

Similar issue applies to `addToAllowlist` and `batchAddToAllowlist` and potentially to `setAllowlistEnabled`.

In other contracts consider similar behavior in:

- `BTCY::updateTransferDenyList`
- `DepositWithdraw::addTokenToWhitelist, removeTokenFromWhitelist, addUserToWhitelist, removeUserFromWhitelist, batchAddUsersToWhitelist, batchRemoveUsersFromWhitelist, setDepositPaused, setWithdrawalPaused`
- `IBTCYHub::setSubscriptionPaused, setRedemptionPaused`

**Aarc:** Fixed in commit [a94fb04](#).

**Cyfrin:** Verified.

### 7.3.3 `ERC4626::deposit, redeem, mint, withdraw` should revert if they would return zero

**Description:** It is good defensive practice to revert if `ERC4626::deposit, redeem, mint, withdraw` would return zero since it never makes sense for these functions to return zero when provided with non-zero inputs; such behavior is only used by blackhats to manipulate vaults commonly via "stealth donation" attacks. Affected contracts:

- `BTCY.sol`

**Aarc:** Fixed in commit [b40ab8a](#).

**Cyfrin:** Verified.

### 7.3.4 Consider enforcing minimum deposit amount

**Description:** BTCY enforces a minimum withdrawal amount but consider also enforcing a minimum deposit amount.

It is good defensive practice to prevent very small deposits and withdrawals (especially 1 wei) since these transaction patterns are not used by legitimate users but are used by blackhats to manipulate vaults. Affected contracts:

- BTCY.sol

**Aarc:** Fixed in commit [30c4800](#).

**Cyfrin:** Verified.

### 7.3.5 Use the `transient` keyword instead of low-level assembly

**Description:** As of Solidity v0.8.30, transient storage is accessible via the `transient` keyword for value types (e.g., `uint256`, `bool`).

Hence declare variables using the `transient` keyword where required instead of using low-level assembly. Affected contracts:

- IBTCY::IS\_FORCE\_TRANSFERRING\_SLOT

**Aarc:** Fixed in commit [529ff75](#).

**Cyfrin:** Verified.

### 7.3.6 Consider automatic allowlist configuration when changing compliance, fee receiver address

**Description:** IBTCY has a compliance address which can effectively seize tokens; presumably it needs to also be able to transfer tokens once seized.

Since IBTCY inherits from AllowList, in IBTCY::setComplianceAddress consider:

- removing the only compliance address from AllowList, and potentially reverting if its balance of tokens is greater than zero
- adding the new compliance address to AllowList

This approach ensures that only the current compliance address is only ever allowed and old addresses don't retain permissions.

A similar improvement could be made to BTCY::setFeeRecipient.

**Aarc:** Acknowledged; keeping allowlist management separate by design for operational flexibility.

### 7.3.7 Rename `Deposit::amountDepositedMinusFees` since subscription fee only used to emit event but not subtracted from amount deposited

**Description:** In IBTCYHub::\_processSubscription the fee is calculated but it is never used or subsequently deducted from the tokens minted to the user:

```
// @audit comment indicates fee only for events, potentially used off-chain
// Calculate subscription fee for accounting (not deducted, just for events)
uint256 feeRate =
    data.operationTypes[i] == OperationType.INSTANT ? $.instantSubscriptionFee :
    ↳ $.regularSubscriptionFee;
// @audit calculated fee never actually used
uint256 fee = _getSubscriptionFees(feeRate, processedAmount);

// Calculate mint amount based on processed subscription amount
// @audit doesn't take into account fee
uint256 ibtcyOwed = _getMintAmountForPrice(processedAmount, data.prices[i]);

// @audit mints full amount to user, doesn't deduct fee, fee never sent anywhere just to emit
↳ event
```

```
// Mint tokens to captured user
$.ibtcy.mint(user, ibtcyOwed);
```

In contrast `_processRedemption` returns fees, `_processRedemptionBatch` sums up the total fees and `processRedemptions` sends them to the fee recipient.

It is also confusing that the storage slot subsequently updated is called `depositor.amountDepositedMinusFees` even though no fee is subtracted:

```
// Update depositor amount
unchecked {
    uint256 remainingDepositAmount = depositorAmount - processedAmount;
    if (remainingDepositAmount != 0) {
        depositor.amountDepositedMinusFees = remainingDepositAmount;
    } else {
        delete $.depositIdToDepositor[depositId];
    }
}
```

If the fee is only for event generation and used off-chain, then consider renaming `Deposit::amountDepositedMinusFees` to `amountDeposited` since fees aren't subtracted from that amount.

**Aarc:** Fixed in commit [f040e83](#).

**Cyfrin:** Verified.

### 7.3.8 IBTCYHub::\_processSubscription should follow Checks-Effects-Interactions pattern by updating depositor amount before making external call to mint tokens

**Description:** `IBTCYHub::_processSubscription` should follow the Checks-Effects-Interactions pattern by updating depositor amount before making an external call to mint tokens:

```
- // Mint tokens to captured user
- $.ibtcy.mint(user, ibtcyOwed);

// Update depositor amount
unchecked {
    uint256 remainingDepositAmount = depositorAmount - processedAmount;
    if (remainingDepositAmount != 0) {
        depositor.amountDepositedMinusFees = remainingDepositAmount;
    } else {
        delete $.depositIdToDepositor[depositId];
    }
}

+ // Mint tokens to captured user
+ $.ibtcy.mint(user, ibtcyOwed);
```

**Aarc:** Fixed in commit [0b4b41e](#).

**Cyfrin:** Verified.

### 7.3.9 Remove obsolete return statements when already using named return variables

**Description:** Remove obsolete return statements when already using named return variables:

- `Pricer::getPriceInfos`

**Aarc:** Fixed in commit [0d0b159](#).

**Cyfrin:** Verified.

### 7.3.10 Duplicate fee helper logic in IBTCYHub can be consolidated

**Description:** IBTCYHub::\_getRedemptionFees and \_getSubscriptionFees implement the same fee calculation logic and differ only by the parameters they're called with.

Consider replacing both helpers with a single internal \_getFees(amount, feeBps) (or similar) and call it from both redemption and subscription paths.

**Aarc:** Fixed in commit [b78b90a](#).

**Cyfrin:** Verified.

### 7.3.11 Inconsistent naming of IBTCYHub::\_processRedemption return value

**Description:** IBTCYHub::\_processRedemption returns (fee, ibtcyToBurn), but IBTCYHub::\_processRedemptionBatch assigns the second value to processedAmount and then sums it into totalIBTCYTokensToBurn. Since "processed amount" typically implies the total (burn + fees), this naming is easy to misinterpret.

Consider renaming the local variable in \_processRedemptionBatch to ibtcyToBurn.

**Aarc:** Fixed in commit [99b97a5](#).

**Cyfrin:** Verified.

### 7.3.12 DepositWithdraw, IBTCYHub, Pricer allow only default admin to revoke role in contrast to the other contracts

**Description:** BTCY and IBTCY override renounceRole, revokeRole to prevent an only DEFAULT\_ADMIN\_ROLE from renouncing their role.

However the DepositWithdraw, IBTCYHub, Pricer contracts don't have a similar restriction; consider adding it if required.

**Aarc:** Fixed in commit [ae685aa](#).

**Cyfrin:** Verified.

### 7.3.13 Fee on transfer tokens not supported

**Description:** The protocol has asked us to examine whether fee on transfer tokens would work with this protocol; currently they are not supported.

However the currently intended token list is WBTC (BitGo), BBTC (Binance), tBTC (Threshold), cbBTC (Coinbase) which don't have a fee on transfer so there is no impact with the intended token list.

**Aarc:** Acknowledged.

### 7.3.14 Consider enhancing BTCY::\_checkIBTCYTransfer to also prevent transfers to address(this)

**Description:** Consider enhancing BTCY::\_checkIBTCYTransfer to also prevent transfers to address(this); if a user sends BTCY shares to address(this), those shares are effectively burned (held by the vault, never redeemable). While this is a user error, the same protective pattern applied for iBTCY could also cover address(this).

**Aarc:** Fixed in commit [3d5932e](#).

**Cyfrin:** Verified.

### 7.3.15 IBTCY::\_checkBTCYVaultTransfer bypassed for mint

**Description:** IBTCY::transfer, transferFrom call to prevent transfers to the BTCY vault, however there is no similar check inside mint enabling tokens to be minted to the BTCY vault.

This allows MINTER\_ROLE to directly mint IBTCY tokens to the BTCY vault which effectively donates assets that inflate BTCY share value.

If this is intended it should be explicitly noted in comments, otherwise it should be prevented.

**Aarc:** Acknowledged; added explicit comment in commit [261ffb7](#).

#### 7.3.16 nonReentrant is not the first modifier on IBTCYHub::processSubscriptions / processRedemptions

**Description:** IBTCYHub::processSubscriptions and processRedemptions apply nonReentrant after other modifiers. While this does not change correctness in Solidity, best practice is to place nonReentrant first to prevent possible reentrancy in other modifiers.

Consider reordering modifiers so nonReentrant is the first modifier on both functions.

**Aarc:** Fixed in commit [a2fb621](#).

**Cyfrin:** Verified.

#### 7.3.17 Zero amount redemption requests can't be cancelled or processed

**Description:** When requesting a subscription or redemption, the input restriction on the amount is that it must be greater or equal to the minimum subscription/redemptions amounts.

However the minimum subscription/redemptions amounts can be configured to zero hence zero amount subscriptions or redemptions can be allowed.

In the case of zero amount subscriptions/redemptions these can't subsequently be cancelled or rescued as IBTCYHub::cancelRedemption, cancelSubscription, rescueRedemption revert if the amount retrieved from storage is zero.

**Recommended Mitigation:** In IBTCYHub::initialize, setMinimumDepositAmount, setMinimumRedemptionAmount enforce that minimum subscription/redemption amounts must be greater than zero.

**Aarc:** Fixed in commit [8d6262e](#).

**Cyfrin:** Verified.

#### 7.3.18 Remove unused errors

**Description:** Consider using or removing the following unused errors:

- IIBTCYHub.sol#L331:

```
error VaultNotWhitelisted();
```

- IIBTCYHub.sol#L340

```
error PriceNotSet();
```

- IPricer.sol#L166

```
error NoPricesSet();
```

- IPricer.sol#L169

```
error PriceNotFound();
```

**Aarc:** Fixed in commit [bc8d3db](#).

**Cyfrin:** Verified.

### 7.3.19 Enforce BTCY vault decimals greater or equal to underlying token decimals

**Description:** For ERC4626 vaults it is good defensive practice to enforce at initialization that the vault's decimals are greater than or equal to the underlying token decimals:

```
function initialize(IIBTCY _ibtcy, address _admin) public initializer {
    require(_admin != address(0), ZeroAddress());

    __ERC20_init("BTCY Vault", "BTCY");
    __ERC4626_init(IERC20(address(_ibtcy)));
+   require(_ibtcy.decimals() <= decimals(), AssetDecimalsTooHigh());
    __AccessControlEnumerable_init();
    __UUPSUpgradeable_init();

    // Grant roles to admin
    _grantRole(DEFAULT_ADMIN_ROLE, _admin);
    _grantRole(PAUSER_ROLE, _admin);
    _grantRole(UPGRADER_ROLE, _admin);
    __AllowList_init(_admin);
}
```

**Aarc:** Fixed in commit [52a5f99](#).

**Cyfrin:** Verified.

### 7.3.20 BTCY::\_checkMinShares should not allow zero totalSupply as it is called after deposits

**Description:** Per finding *"ERC4626::deposit, redeem, mint, withdraw should revert if they would return zero"*, vaults can be manipulated via "stealth donations" where small amounts are deposited to the vault which increase the vault's tokens but not the shares.

The current implementation of BTCY::\_checkMinShares allows this since it allows totalSupply == 0, however since this function is only ever called after a deposit, it shouldn't allow this:

```
function _checkMinShares() internal view {
-   uint256 _totalSupply = totalSupply();
-   require(_totalSupply == 0 || _totalSupply >= MIN_SHARES, MinSharesViolation());
+   require(totalSupply() >= MIN_SHARES, MinSharesViolation());
}
```

Additionally consider removing this function and just enforcing the check inside BTCY::\_deposit:

```
function _deposit(address _caller, address _receiver, uint256 _assets, uint256 _shares)
    internal
    override
    whenNotPaused
{
    super._deposit(_caller, _receiver, _assets, _shares);
-   _checkMinShares();
+   require(totalSupply() >= MIN_SHARES, MinSharesViolation());
}
```

**Aarc:** Fixed in commit [a982611](#).

**Cyfrin:** Verified.

### 7.3.21 User can front-run DoS DepositWithdraw::withdrawTransfer by cancelling redemption request

**Description:** A user can front-run calls to DepositWithdraw::withdrawTransfer by cancelling their redemption request, which makes the entire batch revert.

**Recommended Mitigation:** Can likely just be acknowledged as low risk, not worth adding more complexity to the code. If it ever becomes an issue use a private mempool service like flashbots.

**Aarc:** Acknowledged.

## 7.4 Gas Optimization

### 7.4.1 In Solidity don't initialize to default values

**Description:** In Solidity don't initialize to default values:

```
DepositWithdraw.sol
200:         for (uint256 i = 0; i < users.length; i++) {
213:         for (uint256 i = 0; i < users.length; i++) {

IBTCYHub.sol
163:         $.instantRedemptionFee = 0; // 0%
164:         $.regularRedemptionFee = 0; // 0%
165:         $.instantSubscriptionFee = 0; // 0%
166:         $.regularSubscriptionFee = 0; // 0%
352:         for (uint256 i = 0; i < data.depositIds.length;) {
498:         for (uint256 i = 0; i < data.redemptionIds.length;) {

Pricer.sol
210:         for (uint256 i = 0; i < _priceIds.length;) {

BTCY.sol
178:         for (uint256 i = 0; i < accounts.length; i++) {

base/AllowList.sol
69:         for (uint256 i = 0; i < accounts.length; ++i) {
98:         for (uint256 i = 0; i < accounts.length; ++i) {
```

**Aarc:** Fixed in commit [61ec34d](#).

**Cyfrin:** Verified.

### 7.4.2 Use ReentrancyGuardTransient for faster nonReentrant modifiers

**Description:** Use [ReentrancyGuardTransient](#) for faster nonReentrant modifiers:

```
DepositWithdraw.sol
5:import {ReentrancyGuard} from "openzeppelin-contracts/utils/ReentrancyGuard.sol";
20:contract DepositWithdraw is AccessControlEnumerable, ReentrancyGuard, Pausable, IDepositWithdraw {

IBTCYHub.sol
7:import {ReentrancyGuardUpgradeable} from
↳ "openzeppelin-contracts-upgradeable/utils/ReentrancyGuardUpgradeable.sol";
33:     ReentrancyGuardUpgradeable,
151:     __ReentrancyGuard_init();
```

**Aarc:** Fixed in commit [3a3e350](#).

**Cyfrin:** Verified.

### 7.4.3 Optimize away old value variables when emitting events by emitting events first

**Description:** Optimize away old value variables when emitting events by emitting events first, for example in BTCY::setWithdrawalFee:

```
-         uint256 oldFee = $.withdrawalFee;
+         emit WithdrawalFeeUpdated($.withdrawalFee, fee);
+         $.withdrawalFee = fee;
-         emit WithdrawalFeeUpdated(oldFee, fee);
```

Also affects:

- BTCY::setFeeRecipient, setMinWithdrawAmount

- DepositWithdraw::setTreasury
- IBTCY::setBTCYVault, setComplianceAddress
- IBTCYHub::setInstantRedemptionFee, setRegularRedemptionFee, setInstantSubscriptionFee, setRegularSubscriptionFee, setMinimumDepositAmount, setMinimumRedemptionAmount, setFeeRecipient, setComplianceAddress, setPricer
- Pricer::setPriceFeed

**Aarc:** Fixed in commit [0ee5f5c](#).

**Cyfrin:** Verified.

#### 7.4.4 Use named return variables where this can optimize away local variables

**Description:** Use named return variables where this can optimize away local variables:

- BTCY::withdraw
- Pricer::getPriceIds

**Aarc:** Fixed in commit [7a26d87](#).

**Cyfrin:** Verified.

#### 7.4.5 Remove redundant modifier from BTCY, IBTCY::revokeRole

**Description:** AccessControlUpgradeable::revokeRole already has modifier onlyRole(getRoleAdmin(role)) so there is no need to duplicate it in BTCY::revokeRole:

```
function revokeRole(bytes32 role, address account)
    public
    override(AccessControlUpgradeable, IAccessControl)
-   onlyRole(getRoleAdmin(role))
{
    if (role == DEFAULT_ADMIN_ROLE) {
        require(getRoleMemberCount(DEFAULT_ADMIN_ROLE) > 1, CannotRevokeLastAdmin());
    }
    super.revokeRole(role, account);
}
```

Also affects IBTCY::revokeRole.

**Aarc:** Fixed in commit [79b822f](#).

**Cyfrin:** Verified.

#### 7.4.6 Cache storage to avoid identical storage reads

**Description:** Reading from storage is expensive; cache storage to avoid identical storage reads:

```
DepositWithdraw.sol
// cache `_treasury` in `withdraw`
142:     require(msg.sender == _treasury, NotAuthorized());
147:     IERC20(token).safeTransferFrom(_treasury, user, amount);

Pricer.sol
// cache `$.priceFeed` in `getPrice, getPriceInfo, getPriceInfos`
159:     require(address($.priceFeed) != address(0), PriceFeedNotSet());
161:     (, int256 answer, , uint256 updatedAt, ) = $.priceFeed.getRoundData(uint80(priceId));

182:     require(address($.priceFeed) != address(0), PriceFeedNotSet());
184:     (, int256 answer, , uint256 updatedAt, ) = $.priceFeed.getRoundData(uint80(priceId));
```

```

205:         require(address($.priceFeed) != address(0), PriceFeedNotSet());
211:         (, int256 answer, uint256 updatedAt,) = $.priceFeed.getRoundData(uint80(_priceIds[i]));

231:         require(address($.priceFeed) != address(0), PriceFeedNotSet());
233:         (, int256 answer, uint256 updatedAt,) = $.priceFeed.latestRoundData();

// cache `$.latestPriceId` in `addPrice`, then pass cached value as input to `_addPrice`
// and use that instead of re-reading known value from storage. Same fix in `addCurrentPrice`
261:         if ($.latestPriceId != 0) {
262:             uint256 prev = $.prices[$.latestPriceId].price;
293:             if ($.latestPriceId == 0 || timestamp > $.prices[$.latestPriceId].timestamp) {

351:             if ($.latestPriceId != 0) {
352:                 uint256 prev = $.prices[$.latestPriceId].price;

// cache `$.prices[priceId].price` before `if` check and used cache value in `if` check
311:             if ($.prices[priceId].price == 0) {
314:                 uint256 oldPrice = $.prices[priceId].price;

```

**Aarc:** Fixed in commit [7c57344](#).

**Cyfrin:** Verified.

#### 7.4.7 Better storage packing

**Description:** Better storage packing, assuming already deployed contracts won't be upgraded as these changes require fresh deployment:

- IBTCYHub::IBTCYHubStorage declare subscriptionPaused, redemptionPaused after complianceAddress

**Aarc:** Fixed in commit [abe76b8](#).

**Cyfrin:** Verified.

#### 7.4.8 Redundant DEFAULT\_ADMIN\_ROLE grant in AllowList::\_\_AllowList\_init

**Description:** AllowList::\_\_AllowList\_init grants DEFAULT\_ADMIN\_ROLE to admin, but the calling initializers also grant DEFAULT\_ADMIN\_ROLE to the same admin. This duplicates role setup work during initialization.

Consider removing one of the duplicate DEFAULT\_ADMIN\_ROLE grants (preferably keep the role assignment in a single place, either in \_\_AllowList\_init or in the top-level initializers to avoid redundant storage writes/checks.

**Aarc:** Fixed in commit [93f86f4](#).

**Cyfrin:** Verified.

#### 7.4.9 Remove redundant whenNotPaused modifiers

**Description:** IBTCY::transfer, transferFrom have the whenNotPaused modifier however this is redundant since \_update also has it.

**Aarc:** Fixed in commit [c3172bb](#).

**Cyfrin:** Verified.

#### 7.4.10 Remove event parameters that always have the same value

**Description:** Remove event parameters that always have the same value:

- MintRequested::feesInBTC - always zero (see IBTCYHub::requestSubscription)

**Aarc:** Fixed in commit [7244f81](#).

**Cyfrin:** Verified.

### 7.4.11 More efficient implementations of IBTCYHub::processSubscriptions, processRedemptions

**Description:** More efficient implementations of IBTCYHub::processSubscriptions, processRedemptions, see the before & after:

```
forge test --gas-report
```

|   |                      |       |        |        |        |     |  |
|---|----------------------|-------|--------|--------|--------|-----|--|
| - | processSubscriptions | 10880 | 114636 | 126229 | 166875 | 117 |  |
| + | processSubscriptions | 10880 | 114022 | 125581 | 166161 | 117 |  |
|   |                      |       |        |        |        |     |  |
| - | processRedemptions   | 3691  | 77552  | 89690  | 123840 | 43  |  |
| + | processRedemptions   | 3691  | 77036  | 89038  | 123188 | 43  |  |

Primarily achieved by avoiding copying calldata into memory unnecessarily while still not triggering stack-too-deep errors. The code is also simpler and allows deletion of some structs and helper functions.

```
function processSubscriptions(
    bytes32[] calldata depositIds,
    uint256[] calldata priceIds,
    uint256[] calldata subscriptionAmounts,
    OperationType[] calldata operationTypes
) external override onlyRole(MANAGER_ROLE) nonReentrant whenNotPaused {
    require(depositIds.length == priceIds.length, ArrayLengthMismatch());
    require(depositIds.length == subscriptionAmounts.length, ArrayLengthMismatch());
    require(depositIds.length == operationTypes.length, ArrayLengthMismatch());

    IBTCYHubStorage storage $ = _getIBTCYHubStorage();
    (uint256[] memory prices, uint256[] memory timestamps) = $.pricer.getPriceInfos(priceIds);

    for (uint256 i; i < depositIds.length;) {
        _processSubscription($, depositIds[i], prices[i], subscriptionAmounts[i],
            ↪ operationTypes[i], timestamps[i]);
        unchecked {
            ++i;
        }
    }
}

function _processSubscription(
    IBTCYHubStorage storage $,
    bytes32 depositId,
    uint256 price,
    uint256 subscriptionAmount,
    OperationType operationType,
    uint256 priceTimestamp
) internal {
    Depositor storage depositor = $.depositIdToDepositor[depositId];
    address user = depositor.user;

    // Check if deposit ID is valid
    require(user != address(0), InvalidDepositId());
    require(block.timestamp - priceTimestamp <= MAX_PRICE_AGE, PriceTooOld());

    // Get depositor amount and validate subscription amount
    uint256 depositorAmount = depositor.amountDepositedMinusFees;
    require(
        subscriptionAmount <= depositorAmount,
        SubscriptionAmountCannotBeGreaterThanDeposited(subscriptionAmount, depositorAmount)
    );

    // Calculate subscription fee for accounting (not deducted, just for events)
    uint256 feeRate =
```

```

        operationType == OperationType.INSTANT ? $.instantSubscriptionFee :
        ↪ $.regularSubscriptionFee;
    uint256 fee = _getSubscriptionFees(feeRate, subscriptionAmount);

    // Calculate mint amount based on processed subscription amount
    uint256 ibtcyOwed = _getMintAmountForPrice(subscriptionAmount, price);

    // Mint tokens to captured user
    $.ibtcy.mint(user, ibtcyOwed);

    // Update depositor amount
    unchecked {
        uint256 remainingDepositAmount = depositorAmount - subscriptionAmount;
        if (remainingDepositAmount != 0) {
            depositor.amountDepositedMinusFees = remainingDepositAmount;
        } else {
            delete $.depositIdToDepositor[depositId];
        }
    }

    emit MintCompleted(user, depositId, subscriptionAmount, ibtcyOwed, fee, operationType);
}

function processRedemptions(
    bytes32[] calldata redemptionIds,
    uint256[] calldata priceIds,
    uint256[] calldata redemptionAmounts,
    OperationType[] calldata operationTypes
) external override onlyRole(MANAGER_ROLE) nonReentrant whenNotPaused {
    require(redemptionIds.length == priceIds.length, ArrayLengthMismatch());
    require(redemptionIds.length == redemptionAmounts.length, ArrayLengthMismatch());
    require(redemptionIds.length == operationTypes.length, ArrayLengthMismatch());

    IBTCYHubStorage storage $ = _getIBTCYHubStorage();
    require(!$.redemptionPaused, RedemptionsPaused());

    // Get price information and create batch data
    (uint256[] memory prices, uint256[] memory timestamps) = $.pricer.getPriceInfos(priceIds);

    (uint256 totalFeesInIBTCY, uint256 totalIBTCYTokensToBurn) =
        _processRedemptions($, redemptionIds, prices, redemptionAmounts, operationTypes,
        ↪ timestamps);

    // Transfer fees in iBTCY to feeRecipient
    if (totalFeesInIBTCY != 0) {
        IERC20(address($.ibtcy)).safeTransfer($.feeRecipient, totalFeesInIBTCY);
    }

    // Burn remaining iBTCY held by the hub
    $.ibtcy.burn(totalIBTCYTokensToBurn);
}

// required to avoid stack-too-deep
function _processRedemptions(
    IBTCYHubStorage storage $,
    bytes32[] calldata redemptionIds,
    uint256[] memory prices,
    uint256[] calldata redemptionAmounts,
    OperationType[] calldata operationTypes,
    uint256[] memory timestamps
) internal returns (uint256 totalFeesInIBTCY, uint256 totalIBTCYTokensToBurn) {
    for (uint256 i; i < redemptionIds.length;) {
        (uint256 fee, uint256 processedAmount) = _processRedemption($,

```

```

        redemptionIds[i],
        prices[i],
        redemptionAmounts[i],
        operationTypes[i],
        timestamps[i]);

        totalFeesInIBTCY += fee;
        totalIBTCYTokensToBurn += processedAmount;
        unchecked {
            ++i;
        }
    }
}

function _processRedemption(
    IBTCYHubStorage storage $,
    bytes32 redemptionId,
    uint256 price,
    uint256 redemptionAmount,
    OperationType operationType,
    uint256 priceTimestamp)
    internal
    returns (uint256 feeInIBTCY, uint256 ibtcyToBurn)
{
    // Check if redemption ID is valid
    Redeemer storage redeemer = $.redemptionIdToRedeemer[redemptionId];

    // Capture user address before any modifications
    address user = redeemer.user;
    require(user != address(0), InvalidRedemptionId());
    require(block.timestamp - priceTimestamp <= MAX_PRICE_AGE, PriceTooOld());

    // Validate and process amounts
    uint256 redeemerAmount = redeemer.amountRwaTokenBurned;
    require(
        redemptionAmount <= redeemerAmount,
        RedemptionAmountCannotBeGreaterThanBurned(redemptionAmount, redeemerAmount)
    );

    // Calculate fee and burn amounts
    {
        uint256 feeRate =
            operationType == OperationType.INSTANT ? $.instantRedemptionFee :
            ↪ $.regularRedemptionFee;
        feeInIBTCY = _getRedemptionFees(feeRate, redemptionAmount);
        ibtcyToBurn = redemptionAmount - feeInIBTCY;
    }

    // Update redeemer state
    unchecked {
        uint256 remainingRedemptionAmount = redeemerAmount - redemptionAmount;
        if (remainingRedemptionAmount != 0) {
            redeemer.amountRwaTokenBurned = remainingRedemptionAmount;
        } else {
            delete $.redemptionIdToRedeemer[redemptionId];
        }
    }

    // Emit event with BTC amount for off-chain distribution
    emit RedemptionCompleted(user, redemptionId, ibtcyToBurn,
        _getRedemptionAmountForRwa(ibtcyToBurn, price), feeInIBTCY,
        ↪ operationType);
}

```

**Aarc:** Fixed in commit [b2bad22](#).

**Cyfrin:** Verified.

#### 7.4.12 Remove empty function BTCY::\_checkMinSharesAfterWithdraw

**Description:** Remove empty function BTCY::\_checkMinSharesAfterWithdraw since it is never used; just add its comments inside \_withdraw:

```
function _withdraw(address _caller, address _receiver, address _owner, uint256 _assets, uint256
↳ _shares)
    internal
    override
    whenNotPaused
{
    // snip: unrelated code

    // Transfer fee to fee recipient if fee > 0
    if (fee > 0) {
        address recipient = feeRecipient();
        require(recipient != address(0), FeeRecipientNotSet());
        IERC20(asset()).safeTransfer(recipient, fee);
    }

-     _checkMinSharesAfterWithdraw();
+     // only enforce MIN_SHARES on deposits to stop donation attacks. Skipping the
+     // check here avoids grieving via tiny front-run deposits that would otherwise
+     // make legitimate withdrawals revert when total supply dips below MIN_SHARES.
}
```

**Aarc:** Fixed in commit [8f7015e](#).

**Cyfrin:** Verified.